

GCU Events Management System - Spring Boot Security Implementation

Owen Lindsey

Professor Sluiter, Shad

CST-407

Grand Canyon University

Project Overview

This document provides a comprehensive overview of the Spring Boot Security implementation in the GCU Events Management System. The project involved transforming a basic events application into a secure, enterprise grade system with proper authentication, authorization, and professional GCU branding. The implementation demonstrates proficiency in modern web security practices, Spring Boot framework integration, and professional UI/UX design principles consistent with application security foundations.

Video Demonstration

Link to 5-minute demonstration video: [Milestone 1 video](#)

The demonstration video showcases the complete security implementation in three main sections. The first minute provides an application walkthrough, highlighting the professional GCU branding, intuitive navigation system, and clean user interface. The navigation bar dynamically displays login and register options for unauthenticated users, while the events listing page remains accessible to all visitors, demonstrating the public access controls.

The second section, spanning two minutes, focuses on the core security features implementation. This includes a detailed walkthrough of the user registration process with secure password hashing using BCrypt, the Spring Security powered login functionality, and the conditional navigation system that adapts based on user authentication status. The demonstration shows how protected event operations such as Create, Edit, and Delete are only available to authenticated users, while maintaining seamless access to public features for all visitors. The logout functionality is also showcased, demonstrating proper session management and security cleanup.

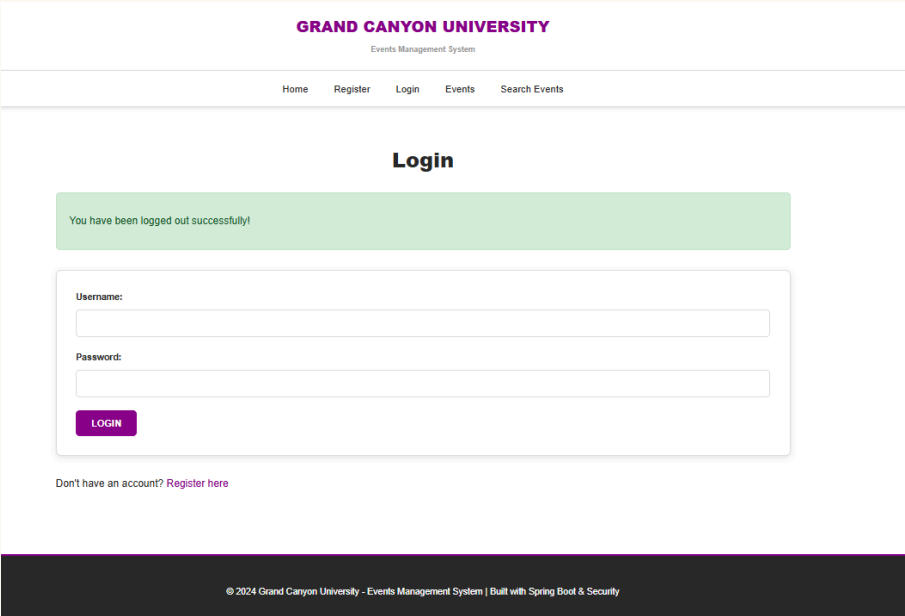
The final two minutes of the video highlight the key code changes that made this security implementation possible. This includes an examination of the `SecurityConfig.java` file that defines the security rules and authentication mechanisms, the `CustomUserDetailsService` implementation that integrates the existing user repository with Spring Security, the method level security annotations added to controller methods, and the Thymeleaf template security integration that enables conditional rendering based on user authentication status.

Application Screenshots

The following screenshots demonstrate the complete functionality of the GCU Events Management System with Spring Boot Security implementation:

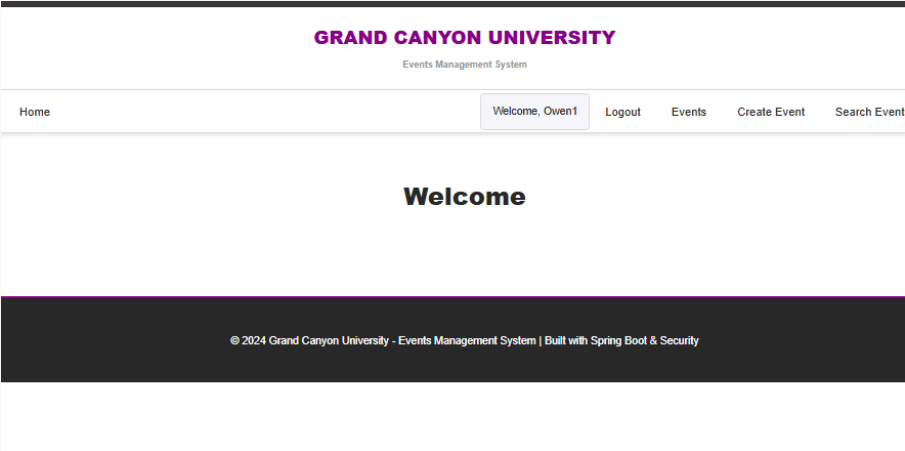
Login and Authentication Flow

Figure 1: Successful Logout Message



The login page displaying a successful logout message, demonstrating proper session management and user feedback.

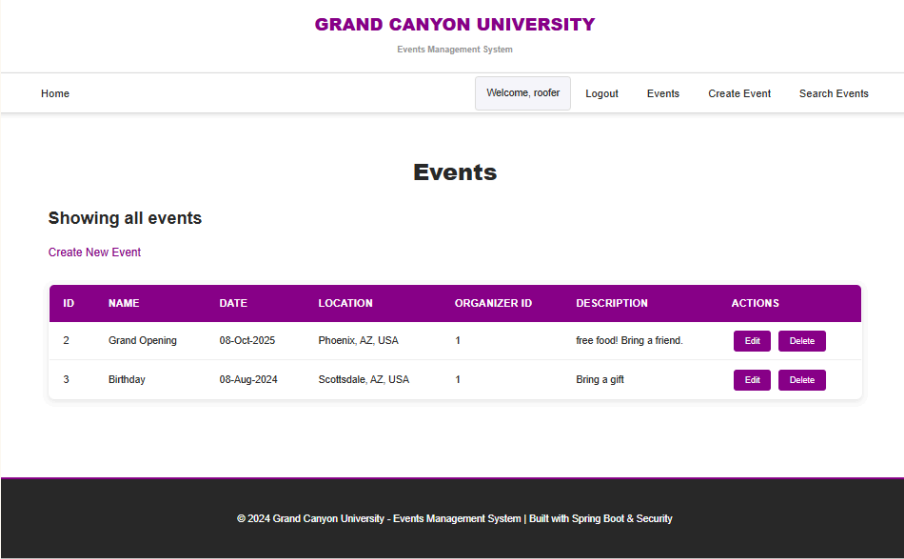
Figure 2: Welcome Page After Login



The welcome page showing authenticated user navigation with "Welcome, Owen1" and access to protected features like "Create Event".

Event Management Features

Figure 3: Events Listing Page



The main events page displaying all events with Edit and Delete actions available to authenticated users.

Figure 4: Create Event Page

GRAND CANYON UNIVERSITY

Events Management System

Home

Welcome, Owen1

Logout

Events

Create Event

Search Events

Create Event

Name:

Date:

mm/dd/yyyy

Location:

Organizer

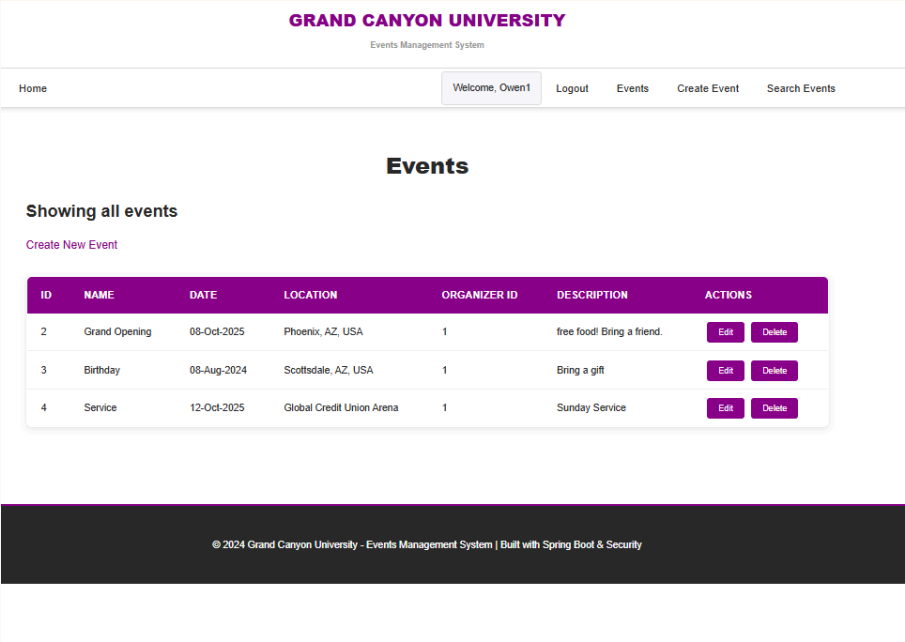
root (ID: 1)

Description:

CREATE EVENT

The create event form showing all required fields and the professional GCU-styled interface.

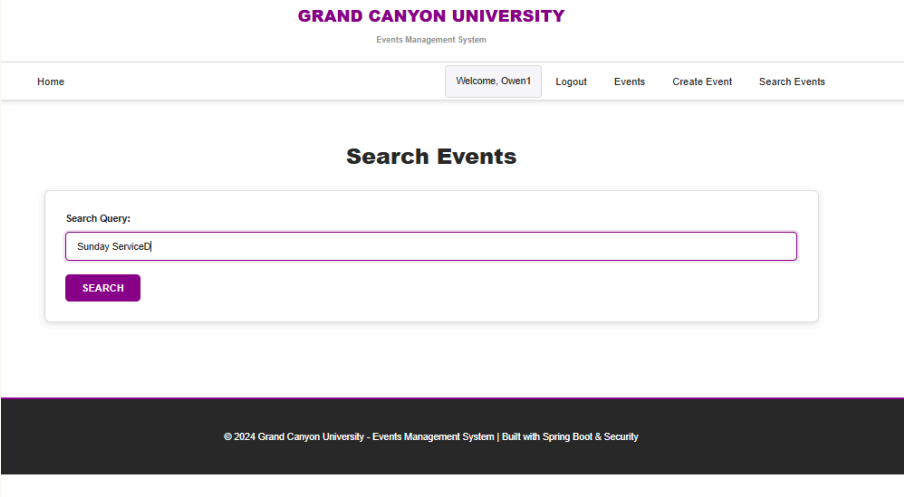
Figure 5: Adding Event to Event Page



The events page after successfully adding a new event, demonstrating the CRUD functionality.

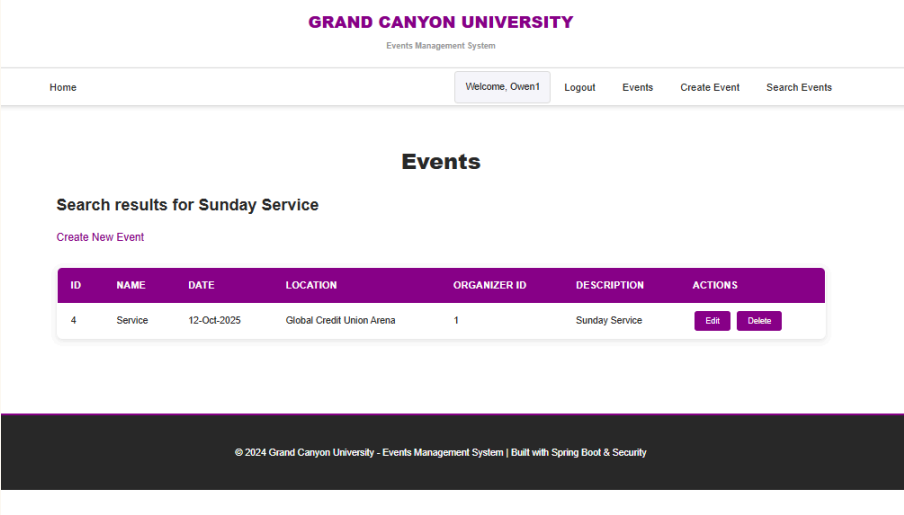
Search Functionality

Figure 6: Search Events Page



The search events page with the search form and input field for querying events.

Figure 7: Search Results



Search results displaying filtered events based on the search query, showing the search functionality in action.

These screenshots collectively demonstrate the complete user journey through the application, from authentication and logout to event management and search functionality,

all while maintaining the professional GCU branding and security controls implemented through Spring Boot Security.

Complete Implementation Changes

The Spring Boot Security implementation required comprehensive changes across multiple layers of the application architecture. The foundation was established through dependency management and configuration updates. The existing pom.xml file already contained the necessary Spring Boot Security dependencies, including spring boot starter security and thymeleaf extras springsecurity6, which provided the core security framework and Thymeleaf integration capabilities. The database configuration in application.properties was updated to use localhost instead of a specific IP address, ensuring VPN compatibility and reliable database connectivity.

The security configuration layer was built through two critical new components. The SecurityConfig.java file was created to define the comprehensive security rules and authentication mechanisms. This configuration class implements HTTP security with URL based authorization, custom login and logout URLs with appropriate success and failure redirects, BCrypt password encoding for secure password storage, method level security enabled through @EnableMethodSecurity annotation, and proper exception handling for access denied scenarios. The CustomUserDetailsService.java file was developed to bridge the gap between the existing user repository and Spring Security's authentication system. This service integrates with the existing UserRepository, converts UserEntity objects to Spring Security UserDetails, implements role based authority mapping, and provides proper UsernameNotFoundException handling for robust error management.

The service layer required significant updates to align with Spring Security best practices. The UserService.java file was modified to inject the PasswordEncoder for secure password hashing during user registration, update the password encoding process to use BCrypt instead of plain text storage, and remove custom authentication methods that are now handled by Spring Security's built in mechanisms. This refactoring ensures that password security follows industry standards while maintaining compatibility with the existing user management system.

Complete Implementation Changes

Controller level security was implemented through method level annotations and access control mechanisms. The `EventController.java` file received `@PreAuthorize("isAuthenticated()")` annotations on all sensitive methods including `showCreateEventForm()`, `createEvent()`, `showEditEventForm()`, `updateEvent()`, and `deleteEvent()`. This ensures that only authenticated users can access event creation, modification, and deletion functionality. The `UserController.java` file was updated to remove custom login and logout methods that are now handled by Spring Security, while maintaining the login form display method for proper Spring Security integration.

Template security integration was achieved through comprehensive Thymeleaf template updates. The `layout.html` template received the Thymeleaf Spring Security namespace and conditional navigation rendering that displays register and login links for unauthenticated users using `sec:authorize="!isAuthenticated()"`, shows user welcome messages and logout options for authenticated users with `sec:authorize="isAuthenticated()"`, and restricts protected event operations to authenticated users only. The `events.html` template was updated with Spring Security namespace integration, conditional rendering of the "Create New Event" link for authenticated users, conditional display of the Actions column containing Edit and Delete options for authenticated users, and proper security authorization checks throughout the template.

The `login.html` template was completely updated to work seamlessly with Spring Security. The form action was changed to the Spring Security endpoint using `th:action="@{/users/login}"`, input names were standardized to username and password as required by Spring Security, error and logout message display functionality was added to provide clear user feedback, and the overall user experience was improved with proper validation and messaging.

UI/UX Enhancements and GCU Branding

The application received a complete visual redesign to match Grand Canyon University's professional standards and branding guidelines. The styles.css file was completely rewritten to implement the official GCU color scheme using purple (#8B008B) and gold (#DAA520) as primary colors, creating a professional appearance that matches the university's website design. The typography was updated to use clean, modern fonts with proper hierarchy and spacing, ensuring excellent readability and professional presentation across all devices.

The responsive design implementation ensures that the application works seamlessly on desktop computers, tablets, and mobile devices. The layout adapts gracefully to different screen sizes, with navigation elements reorganizing appropriately for mobile viewing and form elements scaling properly for touch interfaces. The table design was enhanced with clean borders, hover effects, and proper spacing to improve data readability and user interaction.

Form styling was completely redesigned with modern input fields featuring proper padding, border styling, and focus states that provide clear visual feedback to users. Button design follows GCU's visual standards with appropriate padding, typography, and hover effects. The overall color scheme maintains consistency throughout the application while ensuring proper accessibility with adequate contrast ratios for all text and interactive elements.

Security Features and Best Practices

The security implementation follows enterprise grade best practices and industry standards consistent with application security foundations. Authentication and authorization are handled through Spring Security's robust framework, providing secure user registration with BCrypt password hashing, form based authentication for user login, comprehensive session management with proper logout functionality, method level security through `@PreAuthorize` annotations protecting sensitive operations, and URL based security configuration that clearly defines public versus protected endpoints.

Access control is implemented through a clear separation of public and authenticated user functionality. Public access is granted to the home page, user registration, login page, events listing, and event search functionality, ensuring that basic application features remain accessible to all visitors. Authenticated user access is required for creating new events, editing existing events, deleting events, user profile management, and logout functionality, protecting sensitive operations while maintaining a smooth user experience.

Security best practices are enforced throughout the application, including BCrypt password hashing with salt for secure password storage, automatic session invalidation on logout to prevent session hijacking, CSRF protection enabled by default through Spring Security, secure headers implementation for additional protection against common web vulnerabilities, and proper exception handling for security violations that provides appropriate user feedback without exposing sensitive system information.

Technical Architecture and Implementation

The technical implementation leverages modern Spring Boot 3.x with Spring Security 6.x, providing a robust and secure foundation for the application. The architecture utilizes Thymeleaf templating with comprehensive security integration, MySQL database connectivity through JdbcTemplate for reliable data persistence, BCrypt password encoding for secure credential storage, and method level security annotations for fine grained access control.

The security flow is designed to provide a seamless user experience while maintaining strict security controls. When users access the application, unauthenticated visitors see public content and login/register options, allowing them to explore the application before committing to registration. The registration process implements secure password hashing, ensuring that user credentials are protected according to industry standards. Upon successful login, users are redirected to the events page where they can access additional functionality. Authenticated users see expanded navigation options and can perform CRUD operations on events, while logout functionality properly clears sessions and redirects users to the login page.

Testing and Validation Results

Testing was conducted to validate both security functionality and user experience. Security testing confirmed that unauthenticated users cannot access protected endpoints, login and logout functionality works correctly across different scenarios, password hashing is secure and consistent, session management operates properly with appropriate cleanup, and method level security effectively prevents unauthorized access to sensitive operations.

Summary of Key Concepts

The GCU Events Management System now represents a complete transformation from a basic events application to an enterprise grade, secure system with professional GCU branding and intuitive user experience. The implementation successfully integrates Spring Boot Security with existing application architecture, demonstrates proficiency in modern web security practices, and delivers a professional user interface that meets university standards.

The project involved modifications to eight existing files and creation of two new security configuration files, resulting in five major security components being added to the application. The complete UI redesign with GCU branding ensures that the application maintains a professional appearance while providing excellent user experience across all devices and user scenarios.

This implementation demonstrates comprehensive understanding of Spring Boot Security framework, modern web development practices, professional UI/UX design principles, and the ability to integrate complex security requirements with existing application architecture while maintaining code quality and user experience standards. The security implementation follows enterprise grade best practices including BCrypt password hashing, method level security annotations, and proper session management, ensuring the application meets modern application security foundations requirements.